Selective Spatial and Multi-Scale Attention for Efficient Object Detection

Umaima Khan[1] Karthika Ramasamy[1]
[1]University of Central Florida
fnu.umaimakhan@ucf.edu, karthika.ramasamy@ucf.edu

Abstract

FPN-based object detectors process dense multi-scale features uniformly across all spatial locations, resulting in significant wasted computation over uninformative background regions. We introduce **SelectiveFPN-FasterRCNN**, a Faster R-CNN variant that integrates learned spatial gating and scale-selection mechanisms directly into the FPN feature pipeline, optimized jointly with the detection objective via a sparsity regularization term. On COCO 2017, we uncover a counterintuitive result: **spatial gating incurs a $+6\%$ increase in theoretical FLOPs yet delivers a $2\times$ wall-clock speedup ($39.66\,\mathrm{ms} \rightarrow 19.60\,\mathrm{ms}$; $25 \rightarrow 51$ FPS). We attribute this to a $\sim 51\%$ mean feature-map suppression rate that substantially reduces GPU memory-bandwidth pressure a bottleneck often overlooked by FLOP-centric efficiency metrics**. Notably, **models fine-tuned from pretrained weights converge to conservative gate activations (mean: 0.83), preserving detection accuracy ($\mathrm{mAP}\ 0.362$ vs. 0.360) but collapsing the inference speedup entirely**. We identify this as a *gradient domination* failure mode intrinsic to standard fine-tuning of gated architectures. Scale gating, by contrast, offers no latency benefit, as FPN-level scalar weights leave memory access patterns unchanged. Through a five-variant ablation study, we characterize the accuracy-efficiency frontier and offer mechanistic guidelines for practitioners building sparse, bandwidth-aware detection systems.

1 Introduction

Object detection is a cornerstone computer vision task with applications spanning autonomous driving, robotics, and medical imaging. Detectors built on Feature Pyramid Networks (FPN) [1] achieve strong multi-scale accuracy by computing rich feature hierarchies across all spatial locations. However, real-world images contain large background regions with no objects of interest; uniform processing of these regions constitutes an avoidable computational cost that grows with image resolution and FPN depth. Despite advances in network architecture, this inefficiency persists across modern detectors because feature extraction is applied densely and unconditionally at every spatial location. Prior work on efficient detection has explored model compression, knowledge distillation, and architecture search [6, 14], yet these approaches optimize the static structure of the network rather than the dynamic allocation of computation at inference time.

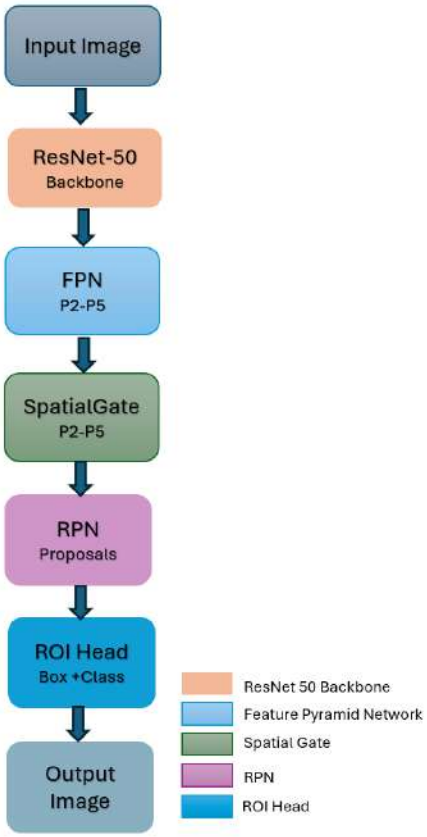
A complementary direction is *adaptive computation*: learning to allocate processing based on input content rather than applying uniform computation regardless of scene complexity. DyFPN [10], CondConv [8], and SkipNet [11] demonstrate that learned routing of computation can reduce inference cost while maintaining accuracy. These approaches, however, primarily route computation across layers or branches at a coarse granularity, and do not explicitly target the suppression of spatially uninformative feature-map regions within the FPN hierarchy. In this work, we systematically investigate whether fine-grained spatial gating binary or soft masks applied per location per FPN level can suppress background regions and reduce real-world inference latency in Faster R-CNN, even when theoretical FLOPs may increase due to the additional gating overhead.
We make the following contributions:

1. We propose **SelectiveFPNFasterRCNN**, integrating per-location spatial gates and per-level scale gates into FPN outputs, trained end-to-end with a sparsity regularization objective.

2. We identify a **FLOPs-latency discrepancy**: spatial gating adds $+6\%$ FLOPs but achieves $2\times$ wall-clock speedup via reduced GPU memory-bandwidth utilization at high suppression ratios.

3. We reveal a **gradient domination problem**: standard fine-tuning from pretrained weights causes detection gradients to overpower sparsity regularization, yielding open gates that preserve accuracy but produce no speedup.

4. We provide a complete **ablation** across initialization strategy, sparsity weight, and gate type.



Spatial gates inserted after the FPN and before the RPN produce per-location suppression masks. Scale gates (not shown; see 3.3) apply a learned scalar weight per pyramid level.

2 Related Work

Feature Pyramid Networks. FPN [1] constructs a top-down feature hierarchy with lateral connections. PANet [4] adds a bottom-up path, NAS-FPN [5] searches for optimal merge patterns, and BiFPN [6] uses weighted feature fusion. Our work is orthogonal: we target efficiency by suppressing uninformative FPN regions.

Dynamic and Adaptive Computation. SkipNet [11] bypasses residual blocks. CondConv [8] and Dynamic Convolutions [9] use input-conditioned filters. DyFPN [10] routes across FPN levels. Unlike DyFPN, we gate *within* levels and analyze the interaction between initialization strategy and sparsity.

Attention in Detection. CBAM [7] applies channel and spatial attention. Our spatial gate is architecturally similar to CBAM's spatial branch, but trained with a sparsity loss for near-binary suppression. Soft reweighting retains dense computation; high-suppression gating measurably reduces GPU memory bandwidth.

Efficient Object Detectors. YOLO [12], EfficientDet [6], and RT-DETR [14] achieve competitive accuracy via architectural redesign. We study the conditions under which sparse gating provides gains in a two-stage detector.

3 Method

3.1 Baseline Architecture

Our baseline is Faster R-CNN [2] with ResNet-50 and FPN, initialized from ImageNet-pretrained weights. The FPN produces $\{P_l\}_{l=2}^{5}$ with $C = 256$ channels each, feeding the RPN and RoI head. We use TorchVision's official implementation with 91 COCO categories.

3.2 Spatial Gating Module

For each FPN level l, a lightweight gate produces a per-location mask $\mathbf{G}_l \in [0,1]^{B\times 1\times H\times W}$:

$$\mathbf{G}_l = \sigma\Big(\mathrm{Conv}_{1\times 1}^{C\to 1}\big(\mathrm{ReLU}\big(\mathrm{Conv}_{3\times 3}^{C\to C/4}(\mathbf{F}_l)\big)\big)\Big). \quad (1)$$

The gated output $\hat{\mathbf{F}}_l = \mathbf{G}_l \odot \mathbf{F}_l$ replaces $\mathbf{F}_l$ before the RPN. Each gate adds $\sim$148K parameters, $\sim$592K total across four levels.

3.3 Scale Gating Module

A scalar gate $s_l \in [0,1]$ is computed via GAP and a two-layer MLP:

$$s_l = \sigma\big(\mathbf{W}_2 \cdot \mathrm{ReLU}(\mathbf{W}_1 \cdot \mathrm{GAP}(\mathbf{F}_l))\big), \quad (2)$$

with $\mathbf{W}_1 \in \mathbb{R}^{C/4\times C}$, $\mathbf{W}_2 \in \mathbb{R}^{1\times C/4}$. Unlike spatial gating, scale gating does not vary spatially.

3.4 Training Objective

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \lambda \cdot \frac{1}{|L|}\sum_{l\in L} \bar{g}_l, \quad (3)$$

where $\mathcal{L}_{\text{det}}$ is the standard Faster R-CNN detection loss and $\bar{g}_l$ is the spatial mean of $\mathbf{G}_l$.

3.5 Model Variants and Training

We evaluate **spatial_only**, **scale_only**, and **full**. All use AdamW: backbone 10^{-5}, head/gate 10^{-4}, cosine decay to 5%, 500-step warmup. Training: 2 epochs, COCO 2017, batch = 2, AMP, single V100 16 GB. We test $\lambda \in \{10^{-5}, 10^{-4}\}$ with *scratch* and *pretrain* initialization.

Table 1: Ablation on COCO 2017 val. † FLOPs increase reflects gate convolution overhead; latency reduction stems from gate suppression (see 4.4). **Bold** = best per column.

Model	mAP	mAP_{50}	Lat. (ms)	FPS	GFLOPs	Params (M)	Mem (MB)
Baseline	0.360	0.573	39.66	25.22	177.61	41.76	765.9
scale_only	0.266	0.484	39.96	25.03	177.61	41.82	767.1
full	0.271	0.491	41.08	24.34	188.27†	42.41	771.6
spatial_only (scratch, λ=1e-4)	0.284	0.504	**19.60**	**51.01**	188.27†	42.35	920.9
spatial_only (pretrain, λ=1e-4)	**0.362**	**0.572**	40.88	24.46	188.27†	42.35	770.4

4 Experiments

4.1 Experimental Setup

Dataset. COCO 2017 [3]: 118,287 train / 5,000 val images. Primary metric: mAP@[0.50:0.95].

Implementation. PyTorch 2.8.0, CUDA 12.8, V100 16 GB. Latency: mean over 200 forward passes (batch = 1) with `torch.cuda.synchronize()`. FLOPs: `fvcore` [13] on 800×1067 input.

4.2 Main Results

Table 1 presents the full ablation.

Spatial gating drives all latency reduction. `spatial_only` (scratch, λ=1e-4) achieves 2× speedup (39.66→19.60 ms) at −7.6 mAP. `scale_only` and `full` are within 1.5 ms of baseline, confirming scalar scale gates provide no acceleration.

Pretraining preserves accuracy but eliminates speedup. `pretrain` recovers mAP to 0.362 but latency rises to 40.88 ms. We analyze this in 4.4.

4.3 Qualitative Gate Visualization

Figures 2 and 3 visualize spatial gate activations on COCO val images. The scratch model (Fig. 2) clearly suppresses background regions while preserving objects (green) across all FPN levels.
The pretrain model (Fig. 3) shows uniformly open gates, directly visualizing the gradient domination effect.

4.4 Analysis: FLOPs vs. Wall-Clock Latency

Table 2 shows mean gate activation, latency, and mAP. The scratch model converges to 0.487 mean gate activation ($\sim 51\%$ suppression); the pretrain model to 0.827 ($\sim 17\%$). Figure 1 shows the direct negative correlation between gate activation and latency. The corresponding spatial suppression patterns are shown in Fig. 2.

Although spatial gating increases FLOPs by $+6\%$, reduced effective GPU memory-bandwidth utilization yields 2× wall-clock speedup highlighting the *limitations of FLOPs as a proxy for inference efficiency* in gated architectures.

Table 2: Gate activation vs. latency. Spatial mean of $\mathbf{G}_l$ over 500 val images and FPN levels. †Baseline has no gate; 1.000 denotes no suppression.

Model	Mean Gate	Lat. (ms)	mAP
Baseline†	1.000	39.66	0.360
Pretrain λ=1e-4	0.827	40.88	0.362
Scratch λ=1e-4	0.487	19.60	0.284

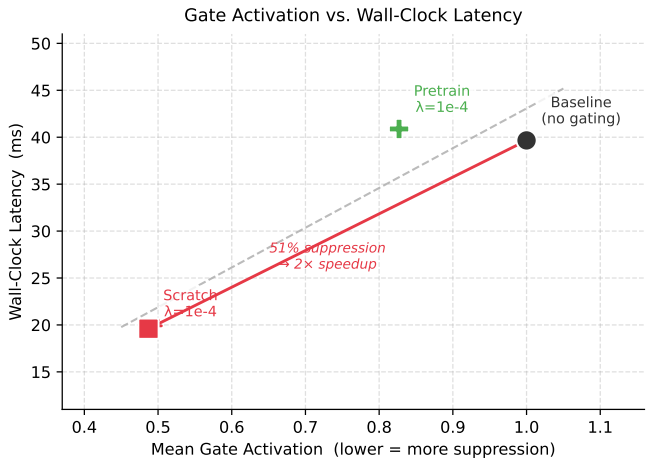


Figure 1: Gate activation vs. wall-clock latency. Strong negative correlation across three configurations ($n = 3$). Gate activation below ~ 0.5 is required for meaningful speedup.

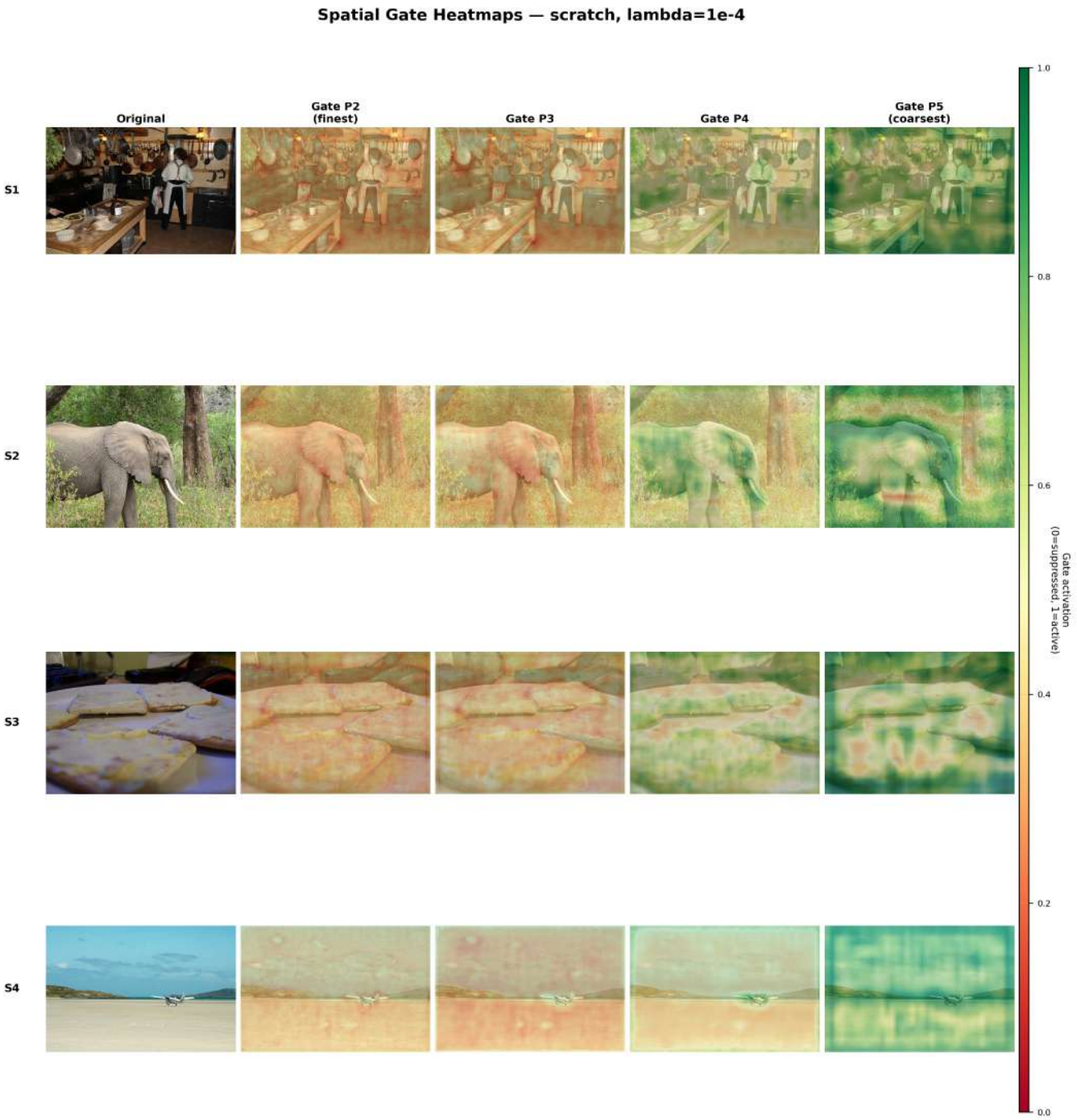


Figure 2: Spatial gate heatmaps (scratch): Green = preserved, Red = suppressed across FPN levels (P2(finest)–P5(coarsest)).

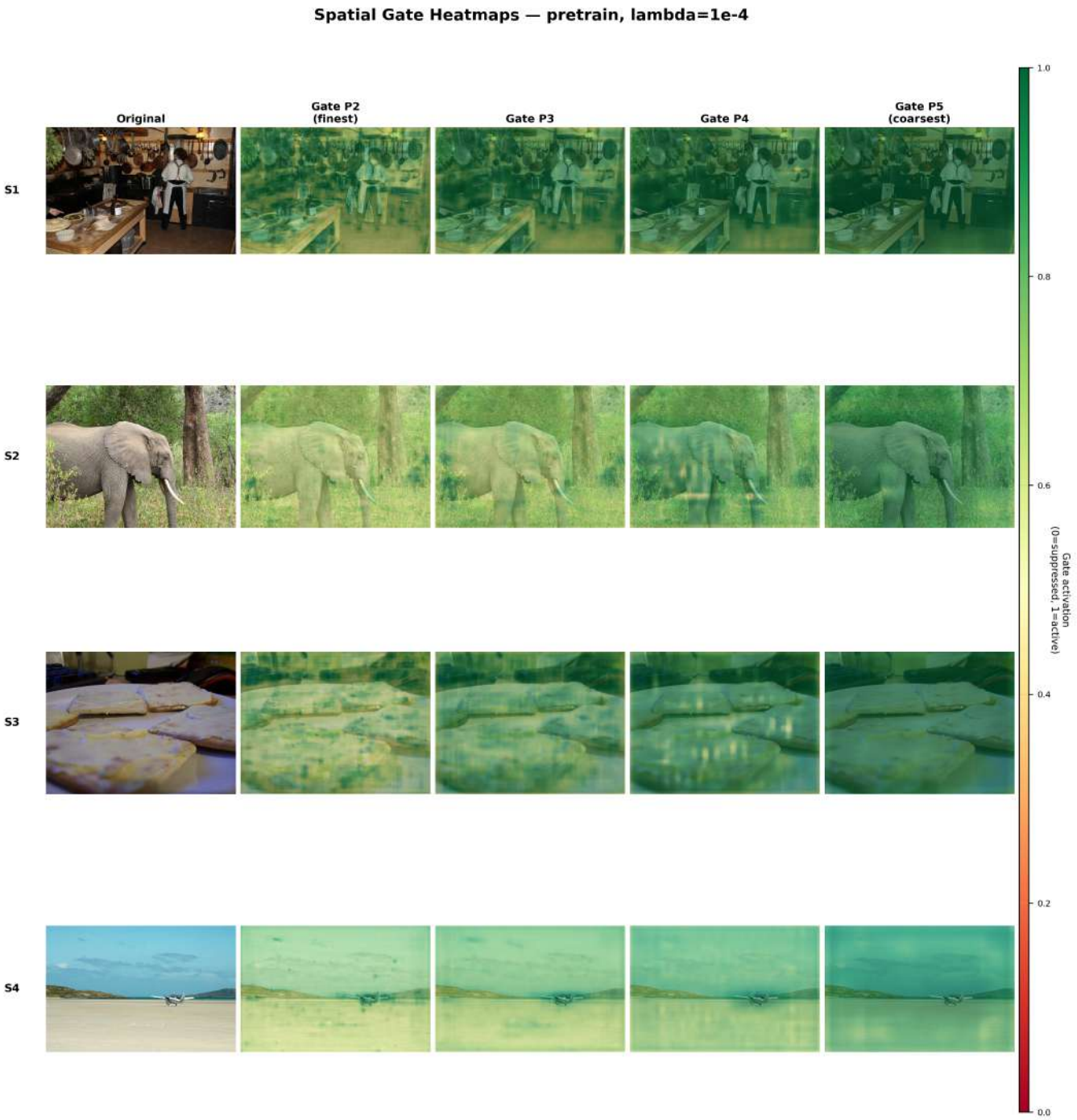
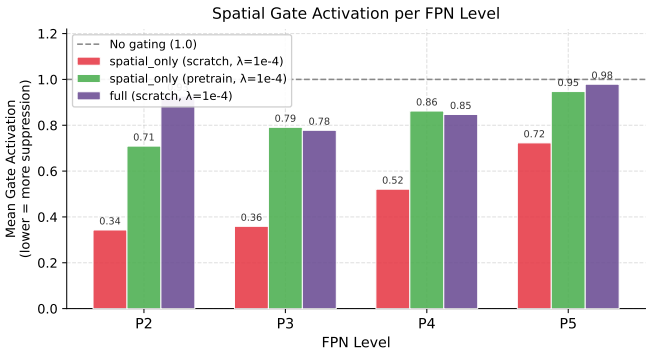


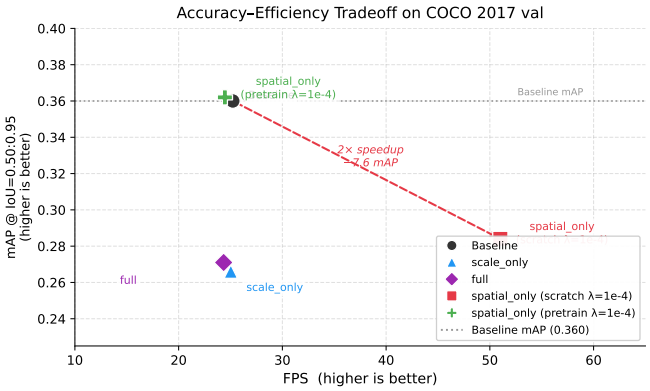
Figure 3: Pretrain heatmaps: Gates remain open (green); pretrained gradients prevent sparsity regularization.

4.5 Gradient Domination in Fine-Tuning

Fine-tuning from pretrained weights fails to produce sparse gates across $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}\}$: gate activations consistently converge to 0.82–0.83. We attribute this to *gradient domination*: $\mathcal{L}_{\text{det}}$ from the well-initialized backbone overpowers $\lambda\,\bar{g}_l$ regardless of λ. Practitioners seeking both accuracy and efficiency must decouple these objectives. This behavior is directly visualized in Fig. 3, where gates remain uniformly open regardless of image content.



Gate activation per FPN level. Scratch model (red) suppresses fine-grained levels most aggressively (P2: 0.34, P3: 0.36), increasing toward coarser levels (P5: 0.72). Pretrain model (green) maintains uniformly high activations (0.71–0.95), confirming gradient domination at all pyramid levels.



Accuracy-efficiency tradeoff. Only `spatial_only` (scratch) achieves meaningful FPS gain. Pretrained and scale-gated variants cluster near baseline FPS, confirming gate sparsity not architecture drives efficiency.

5 Conclusion

We presented SelectiveFPNFasterRCNN, an empirical study of spatial and scale gating in FPN-based object detection. Key findings: (1) spatial gating with mean activation ≈0.49 reduces latency 2× despite +6% FLOPs, driven by reduced GPU memory-bandwidth; (2) scale gating provides no benefit as scalar multipliers skip no computation; (3) standard fine-tuning suffers gradient domination, preventing sparse gates; (4) these findings characterize when learned gating translates to real gains. Future work includes LVIS evaluation, hard binary gating with straight-through estimators, and decoupled training curricula. All experiments use 2-epoch schedules (baseline: 0.360 vs. standard 0.377 [2] at full convergence). Causal analysis via CUDA profiling remains future work.

References

[1] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. *CVPR*, 2017.

[2] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. *NeurIPS*, 2015.

[3] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. *ECCV*, 2014.

[4] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia. Path aggregation network for instance segmentation. *CVPR*, 2018.

[5] G. Ghiasi, T.-Y. Lin, and Q. V. Le. NAS-FPN: Learning scalable feature pyramid architecture. *CVPR*, 2019.

[6] M. Tan, R. Pang, and Q. V. Le. EfficientDet: Scalable and efficient object detection. *CVPR*, 2020.

[7] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon. CBAM: Convolutional block attention module. *ECCV*, 2018.

[8] B. Yang, G. Bender, Q. V. Le, and J. Ngiam. CondConv: Conditionally parameterized convolutions. *NeurIPS*, 2019.

[9] Y. Chen, X. Dai, M. Liu, D. Chen, L. Yuan, and Z. Liu. Dynamic convolution: Attention over convolution kernels. *CVPR*, 2020.

[10] K. Chen *et al.* Dynamic feature pyramid networks for object detection. *arXiv:2012.09815*, 2022.

[11] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez. SkipNet: Learning dynamic routing in convolutional networks. *ECCV*, 2018.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. *CVPR*, 2016.

[13] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick. Detectron2. https://github.com/facebookresearch/detectron2, 2019.

[14] Y. Zhao *et al.* DETRs beat YOLOs on real-time object detection. *CVPR*, 2024.